МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ ГОМЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ П. О. СУХОГО

Факультет автоматизированных и информационных систем

Кафедра «Информатика»

Специальность 1-40 04 01 «Информатика и технологии программирования»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к дипломному проекту

на тему: «**Платформа онлайн-голосования в системе образования**»

Исполнитель: студент гр. ИП-41 Загорцев В. В.

Руководитель: Прокопенко Д. В.

Дата проверки: _____

Дата допуска к защите: _____

Дата защиты: _____

Оценка работы: _____

Подписи членов комиссии

По защите дипломной работы: _____

Гомель 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 5
1.1 Анализ предметной области 5
1.2 Аналитический обзор существующих аналогов 6
1.3 Обзор технологий для реализации программного обеспечения 7
1.4 Анализ инструментальных средств автоматизации разработки и тестирования 9
1.5 Постановка задачи на дипломное проектирование 12
2 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 13
2.1 Общая структура приложения 13
2.2 Функциональная модель программного комплекса 13
2.3 Информационная модель программного комплекса 13

ВВЕДЕНИЕ

В условиях цифровизации общества многие процессы принятия коллективных решений переходят в электронную форму. Одной из таких сфер является организация голосований. Онлайн-голосование упрощает участие пользователей в принятии решений, снижает организационные затраты и ускоряет обработку результатов. Однако традиционные электронные системы голосования имеют ряд ограничений, связанных с доверием, прозрачностью и безопасностью.

Основной проблемой централизованных систем является необходимость доверия к организации, управляющей голосованием. Пользователи не всегда могут убедиться в корректности подсчёта голосов и в том, что их голос был учтён без изменений. Кроме того, централизованные базы данных уязвимы к несанкционированному доступу и модификации информации. В ряде случаев голосующие принадлежат к распределённым сообществам, где отсутствует единый центр управления, а решения принимаются совместно. В таких сценариях традиционные централизованные системы не позволяют эффективно обеспечивать прозрачность и контроль со стороны всех участников.

Одним из перспективных решений является использование технологии блокчейн. Она позволяет создавать децентрализованные системы голосования, в которых каждый голос фиксируется и не может быть изменён после записи, а также обеспечивается возможность независимой проверки корректности подсчёта голосов каждым участником. Такая архитектура гарантирует прозрачность, защиту данных и доверие к результатам даже в распределённых сообществах.

Целью данной работы является разработка децентрализованной платформы онлайн-голосования на основе блокчейн-технологии, обеспечивающей прозрачность, безопасность и возможность независимой проверки результатов.

В рамках дипломной работы будет разработана информационная модель платформы, включающая основные сущности системы: пользователей, голосования, варианты выбора и результаты. Особое внимание будет уделено смарт-контрактам, обеспечивающим регистрацию голосов и корректный подсчёт результатов.

Результатом работы станет программный комплекс платформы онлайн-голосования, обеспечивающий прозрачность проведения голосований, защиту данных пользователей и возможность проверки результатов. Разработанная система может применяться в образовательных учреждениях, корпоративных организациях, онлайн-сообществах и других сферах, где требуется проведение честных и безопасных процедур коллективного голосования.

Задачами дипломной работы являются:

– изучение особенностей разработки современных Web-приложений и клиент-серверных архитектур;

– проведение анализа технологии блокчейн и возможностей её применения в сфере онлайн-голосований;

– разработка архитектуры Web-приложения для проведения голосований с учётом различных сценариев: студенческие выборы, онлайн-сообщества и

конкурсы/хакатоны;
– проектирование структуры базы данных и механизм взаимодействия с блокчейн-смарт-контрактами для регистрации и проверки голосов;
– реализация программных модулей приложения, включая пользовательский интерфейс, систему регистрации и подсчёта голосов, интеграцию с уведомлениями и смарт-контрактами;
– проведение тестирования разработанного программного обеспечения, проверить корректность учёта голосов и оценить работоспособность платформы в разных сценариях голосований.

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Анализ предметной области

Процедура голосования является одним из основных механизмов принятия коллективных решений в различных сферах деятельности. Голосование используется в государственных структурах, корпоративном управлении, образовательных учреждениях, общественных организациях и других системах, где требуется выражение мнения группы участников и определение итогового решения на основе большинства голосов.

С развитием информационных технологий широкое распространение получили системы электронного голосования. Электронное голосование представляет собой процесс подачи и подсчёта голосов с использованием информационных систем и сетевых технологий. Такие системы позволяют автоматизировать процедуру голосования, ускорить обработку результатов и обеспечить участие пользователей независимо от их географического расположения.

Онлайн-платформы голосования активно применяются в различных областях. Например, они используются при проведении корпоративных опросов, голосований в образовательных учреждениях, выборах органов студенческого самоуправления, принятии решений в онлайн-сообществах и организациях, а также при проведении конкурсов, хакатонов и научных конференций. Использование цифровых платформ позволяет существенно повысить удобство участия пользователей и сократить организационные расходы.

Однако существующие системы электронного голосования в большинстве случаев основаны на централизованной архитектуре. Это означает, что все данные о голосовании, включая информацию о пользователях и их голосах, хранятся на серверах одной организации или оператора системы. Подобный подход вызывает ряд проблем, связанных с безопасностью, прозрачностью и доверием пользователей.

Одной из ключевых проблем централизованных систем является возможность изменения или удаления данных администраторами системы либо злоумышленниками при получении доступа к серверу. Пользователи не всегда имеют возможность убедиться в том, что их голос был корректно зафиксирован и учтён при подсчёте результатов. В ряде случаев участники голосования принадлежат к распределённым сообществам, где отсутствует единый центр управления, а решения принимаются совместно. В таких сценариях традиционные централизованные системы не обеспечивают прозрачность и контроль со стороны всех участников.

Другой важной проблемой является проверяемость процедуры голосования. Участники должны иметь возможность самостоятельно убедиться в корректности проведения голосования и в том, что результаты получены без манипуляций. В традиционных электронных системах такие механизмы часто отсутствуют или реализованы недостаточно эффективно.

В связи с перечисленными проблемами возрастает интерес к использованию децентрализованных технологий для реализации систем голосования. Одним из наиболее перспективных решений является применение технологии блокчейн. Блокчейн

представляет собой распределённую систему хранения данных, в которой информация записывается в виде последовательности блоков и защищается с использованием криптографических методов. Основным преимуществом блокчейна является невозможность изменения ранее записанных данных без согласия участников сети.

Использование блокчейн-технологии позволяет обеспечить прозрачность процесса голосования, повысить уровень доверия пользователей к системе и предоставить возможность независимой проверки результатов. Каждый голос фиксируется в блокчейне в виде транзакции или хэша, что обеспечивает неизменяемость данных и позволяет участникам проверить факт учёта своего голоса. Такая архитектура особенно актуальна для распределённых сообществ и сценариев, где отсутствует единый центр управления, а решения принимаются совместно.

Таким образом, анализ предметной области показывает, что существующие системы электронного голосования обладают рядом ограничений, связанных с централизованным хранением данных и недостаточной прозрачностью процедур. Применение блокчейн-технологий и смарт-контрактов позволяет устранить часть этих проблем и создать более надёжные, прозрачные и проверяемые системы онлайн-голосования.

1.2 Аналитический обзор существующих аналогов

Добавить фотки и пересмотреть

С развитием цифровых технологий появилось большое количество систем электронного голосования, предназначенных для проведения опросов, выборов и принятия коллективных решений. Такие платформы позволяют автоматизировать процесс голосования, упростить участие пользователей и ускорить обработку результатов. Однако большинство существующих решений имеют централизованную архитектуру и обладают рядом ограничений, связанных с безопасностью, прозрачностью и возможностью независимой проверки результатов.

Одной из наиболее известных платформ онлайн-голосования является система SurveyMonkey. Данная платформа предназначена для проведения опросов и голосований в сети Интернет. Система предоставляет пользователям удобный интерфейс для создания опросов, формирования вариантов ответов и анализа результатов. Пользователи могут участвовать в голосовании через веб-интерфейс или по ссылке, предоставленной организатором. Несмотря на удобство использования и широкие функциональные возможности, данная система является полностью централизованной. Все данные хранятся на серверах компании, а пользователи не имеют возможности проверить неизменность результатов голосования.

Другим распространённым сервисом является Google Forms, который также используется для проведения опросов и голосований. Платформа позволяет быстро создавать формы для голосования, распространять их среди участников и автоматически собирать результаты. Одним из преимуществ системы является простота использования и интеграция с другими сервисами. Однако данная платформа также не обеспечивает

прозрачности процедуры голосования. Участники не имеют возможности проверить, был ли их голос учтён корректно, а результаты полностью контролируются владельцем формы.

Отдельную категорию составляют специализированные системы электронного голосования, предназначенные для проведения официальных или корпоративных выборов. Примером такой системы является Helios Voting, которая используется для проведения защищённых онлайн-выборов. Данная система использует криптографические методы для обеспечения анонимности и проверки голосов. Helios предоставляет возможность независимой проверки результатов голосования, что повышает доверие участников к системе. Однако архитектура данной платформы остаётся централизованной, а управление системой осуществляется ограниченным кругом администраторов.

В последние годы активно развиваются системы голосования, использующие технологию блокчейн. Одним из примеров является платформа Voatz, предназначенная для проведения удалённых голосований с использованием мобильных устройств. В системе используется блокчейн для хранения информации о голосах и повышения прозрачности процедуры. Однако такие решения пока имеют ограниченное распространение и часто требуют специализированной инфраструктуры или мобильных приложений.

Проведённый анализ существующих систем показывает, что большинство современных платформ онлайн-голосования ориентированы на удобство использования и быстроту проведения опросов, однако не всегда обеспечивают достаточный уровень прозрачности и проверяемости результатов. Централизованное хранение данных создаёт потенциальные риски изменения информации, а также снижает уровень доверия пользователей к итогам голосования.

В связи с этим возникает необходимость разработки систем голосования, основанных на децентрализованных технологиях. Использование блокчейна и смарт-контрактов позволяет обеспечить неизменяемость записей, прозрачность процедуры голосования и возможность независимой проверки результатов всеми участниками. Разработка такой платформы позволит повысить уровень доверия пользователей к системам онлайн-голосования и расширить область их применения.

1.3 Обзор технологий для реализации программного обеспечения

Современные *Web*-приложения разрабатываются с использованием разнообразных технологий и инструментов, которые позволяют создавать масштабируемые, надёжные и безопасные информационные системы. Для реализации системы онлайн-голосований, обеспечивающей прозрачность и возможность независимой проверки результатов, был выбран комплекс технологических решений, включающий клиентскую часть, серверную часть, систему управления базами данных и блокчейн-технологии.

Архитектура системы построена по принципу клиент-серверной модели: пользователь взаимодействует с интерфейсом веб-приложения через браузер, при этом клиентская часть отвечает за отображение интерфейса и обработку действий

пользователя, а серверная часть обеспечивает обработку запросов, хранение данных и реализацию бизнес-логики. Взаимодействие между компонентами осуществляется через сетевые протоколы передачи данных и специализированные *API*-интерфейсы.

JavaScript является одним из ключевых языков программирования для разработки *Web*-приложений. Он применяется для создания интерактивных интерфейсов, обработки событий и динамического обновления содержимого страниц. Основными особенностями языка являются динамическая типизация, автоматическое управление памятью и поддержка асинхронного программирования. С помощью технологии *AJAX JavaScript* обеспечивает обмен данными между клиентом и сервером без полной перезагрузки страницы, что повышает скорость работы интерфейса и удобство взаимодействия с пользователем.

Для построения пользовательского интерфейса используется библиотека *React*, которая широко применяется в современных *Web*-приложениях. *React* поддерживает компонентный подход: интерфейс делится на отдельные компоненты, каждый из которых реализует определённую часть функциональности системы. Такой подход облегчает разработку, повышает повторное использование кода и упрощает поддержку программного обеспечения. Механизм *Virtual DOM*, используемый в *React*, позволяет оптимизировать обновление интерфейса, изменяя только те элементы, которые действительно были изменены, что повышает производительность приложения.

Серверная часть разрабатывается на платформе *ASP.NET*, входящей в экосистему *Microsoft .NET*, и предназначенной для создания Web-приложений, *API*-сервисов и распределённых систем. Основным языком разработки является C#, современный объектно-ориентированный язык со строгой типизацией. *ASP.NET* предоставляет инструменты для обработки *HTTP*-запросов, управления пользовательскими данными, реализации механизмов безопасности и интеграции с базами данных.

Серверная часть реализует *REST API* (*Representational State Transfer Application Programming Interface*), обеспечивающее взаимодействие между клиентской и серверной частями приложения. В рамках *REST API* используются стандартные *HTTP*-методы: *GET* – получение данных; *POST* – создание новых записей; *PUT* – обновление существующих данных; *DELETE* – удаление данных. Передача данных между клиентом и сервером осуществляется в формате *JSON*, что обеспечивает гибкость архитектуры и возможность масштабирования системы.

Для хранения данных используется система управления базами данных *PostgreSQL*, которая обеспечивает надёжное хранение информации, поддержку транзакций и возможность выполнения сложных запросов. В базе данных хранятся сведения о пользователях, голосованиях, кандидатах или вариантах выбора, результатах голосований и другие необходимые данные системы.

Для обеспечения безопасного доступа к системе применяется механизм *JWT* (*JSON Web Token*). После успешной авторизации сервер формирует токен, который содержит зашифрованную информацию о пользователе и его правах доступа. *JWT* состоит из трёх частей: *Header* (заголовок), *Payload* (полезная нагрузка) и *Signature* (подпись).

– *header* содержит информацию о типе токена и используемом алгоритме подписи;

– *payload* включает данные о пользователе и его правах доступа, а также дополнительные сведения, например время жизни токена;

– *signature* формируется на основе заголовка и полезной нагрузки с использованием секретного ключа или пары ключей (в случае асимметричной криптографии), что обеспечивает целостность и защиту от подделки токена.

Токен сохраняется на стороне клиента и передаётся серверу при выполнении последующих запросов. Сервер проверяет подпись и расшифровывает *payload* для определения прав доступа пользователя, что позволяет безопасно управлять доступом к ресурсам системы без необходимости хранения сессий на сервере.

Для обеспечения прозрачности и проверяемости голосований используется технология блокчейн на базе сети *Ethereum*. *Ethereum* представляет собой децентрализованную платформу для создания и выполнения смарт-контрактов. Информация о голосах хранится в распределённом реестре блокчейн-сети, который состоит из последовательности блоков, связанных криптографически. После добавления данных в блокчейн изменить или удалить их невозможно, что обеспечивает высокий уровень безопасности и доверие пользователей к результатам голосований.

Solidity используется для разработки смарт-контрактов, которые автоматически выполняются при наступлении заданных условий. Смарт-контракты реализуют регистрацию участников голосования, фиксацию голосов, защиту от повторного голосования и автоматический подсчёт результатов. После публикации смарт-контракта его код становится неизменяемым, что гарантирует соблюдение правил работы системы.

Для взаимодействия пользователей с блокчейн-сетью используется криптокошелёк *MetaMask*, позволяющий безопасно подписывать транзакции и фиксировать голоса в блокчейне. Клиентская часть приложения подключается к кошельку и может выполнять операции: подключение к кошельку, отправка транзакций голосования, получение информации о выполненных действиях и чтение данных из смарт-контрактов.

Использование библиотек *Web3.js* или *Ethers.js* обеспечивает удобную интеграцию *Web*-приложения с блокчейн-инфраструктурой и позволяет клиентской части напрямую взаимодействовать со смарт-контрактами.

Таким образом, комплексное применение современных технологий, включая клиентскую и серверную части, базы данных, блокчейн и смарт-контракты, обеспечивает разработку системы онлайн-голосований, обладающей высокой степенью безопасности, прозрачности, проверяемости и удобства для всех участников процесса.

1.4 Анализ инструментальных средств автоматизации разработки и тестирования

Для разработки программного комплекса системы онлайн-голосований будет использован комплекс современных инструментов и технологий, обеспечивающих удобство разработки, безопасное взаимодействие с блокчейном и управление данными.

Применение специализированных средств повышает эффективность работы, облегчает управление проектом и обеспечивает более высокое качество программного продукта.

В качестве основной среды разработки выбран *JetBrains Rider*, а для работы с контейнеризацией и развёртыванием сервисов используется *Docker*. Хранение и работа с исходным кодом осуществляется через *Git* и платформу *GitHub*. Для управления реляционными базами данных применяется *DataGrip*, а тестирование *API* выполняется с помощью *Postman*. Взаимодействие с блокчейн-сетью реализовано через криптокошелёк *MetaMask* и инструменты разработки смарт-контрактов *Hardhat* с сетью тестирования *Sepolia*. Для проектирования и визуализации архитектуры системы используется *StarUML*.

JetBrains Rider представляет собой современную интегрированную среду разработки (*IDE*) для создания приложений на платформе *.NET*, разработанную компанией *JetBrains*. Она предоставляет широкий набор инструментов для программирования на *C#,* включая разработку серверной части *Web*-приложений на *ASP.NET*, реализацию *REST API*, работу с базами данных и бизнес-логикой. Основные возможности *Rider* включают интеллектуальное автодополнение кода, встроенную отладку, анализ и поиск ошибок, интеграцию с *Git*, поддержку *Docker* и мощные инструменты рефакторинга. Высокая производительность и удобный интерфейс позволяют эффективно работать с крупными проектами, а наличие множества плагинов расширяет функциональность *IDE*.

Docker является платформой для контейнеризации, обеспечивающей запуск приложений в изолированных контейнерах со всеми необходимыми зависимостями. Это упрощает развертывание и тестирование системы онлайн-голосований, позволяя контейнерам работать одинаково на разных серверах и операционных системах. Применение *Docker* обеспечивает изоляцию приложений, упрощает масштабирование, повышает стабильность работы и облегчает процесс деплоя. В проекте *Docker* используется для контейнеризации серверной части и базы данных, гарантируя стабильную работу приложения.

Для управления версиями исходного кода в процессе разработки используется распределённая система контроля версий *Git*. Данный инструмент позволяет отслеживать изменения в программном коде, фиксировать этапы разработки, а также при необходимости возвращаться к предыдущим версиям проекта. Применение *Git* обеспечивает удобную работу с ветками разработки, что позволяет параллельно реализовывать различные функциональные возможности системы и впоследствии объединять внесённые изменения.

Для удалённого хранения репозитория и организации совместной работы над проектом используется платформа *GitHub*, которая тесно интегрируется с системой контроля версий *Git*. Платформа предоставляет инструменты для размещения исходного кода в удалённом репозитории, управления ветками, отслеживания изменений и взаимодействия между разработчиками. Использование *GitHub* обеспечивает безопасное хранение программного кода, упрощает процесс коллективной разработки и способствует эффективному управлению жизненным циклом проекта.

DataGrip, разработанный компанией *JetBrains*, предоставляет графический интерфейс для работы с базами данных и выполнения *SQL*-запросов. Он поддерживает различные СУБД, включая *PostgreSQL*, применяемую в данном проекте. Основные

возможности *DataGrip* включают выполнение запросов, визуальное управление таблицами, анализ структуры базы данных, автодополнение *SQL*-кода и редактирование данных, что облегчает разработку и администрирование базы данных приложения.

Postman позволяет тестировать *REST API*, отправляя *HTTP*-запросы к серверу и анализируя ответы. Он поддерживает различные методы запросов (*GET*, *POST*, *PUT*, *DELETE*), передачу данных в формате *JSON*, проверку авторизации и аутентификации, создание коллекций запросов и автоматизацию тестирования. Использование *Postman* ускоряет процесс разработки серверной части приложения.

MetaMask выступает криптовалютным кошельком для взаимодействия пользователей с блокчейн-сетью *Ethereum*. Он используется для подписания транзакций голосов в сети *Ethereum*, позволяя безопасно работать с блокчейн-инфраструктурой без хранения приватных ключей на сервере.

Hardhat – это инструмент для разработки, тестирования и развертывания смарт-контрактов в сети *Ethereum*. Он позволяет компилировать контракты на *Solidity*, запускать локальную тестовую сеть *Ethereum*, автоматизировать тестирование и деплой контрактов в тестовые и публичные сети. Одним из ключевых преимуществ *Hardhat* является возможность локального тестирования смарт-контрактов без совершения реальных транзакций, что существенно упрощает процесс разработки блокчейн-компонентов системы онлайн-голосований.

StarUML – это приложение для создания *UML*-диаграмм, которое позволяет разработчикам создавать модели проектов, планировать архитектуру и дизайн приложений. *StarUML* имеет графический интерфейс пользователя, который предоставляет множество инструментов для создания и редактирования диаграмм, включая диаграммы классов, диаграммы последовательностей, диаграммы состояний, диаграммы активностей и многое другое. На рисунке 1.1 в виде выпадающего меню представлен весь список возможных диаграмм.

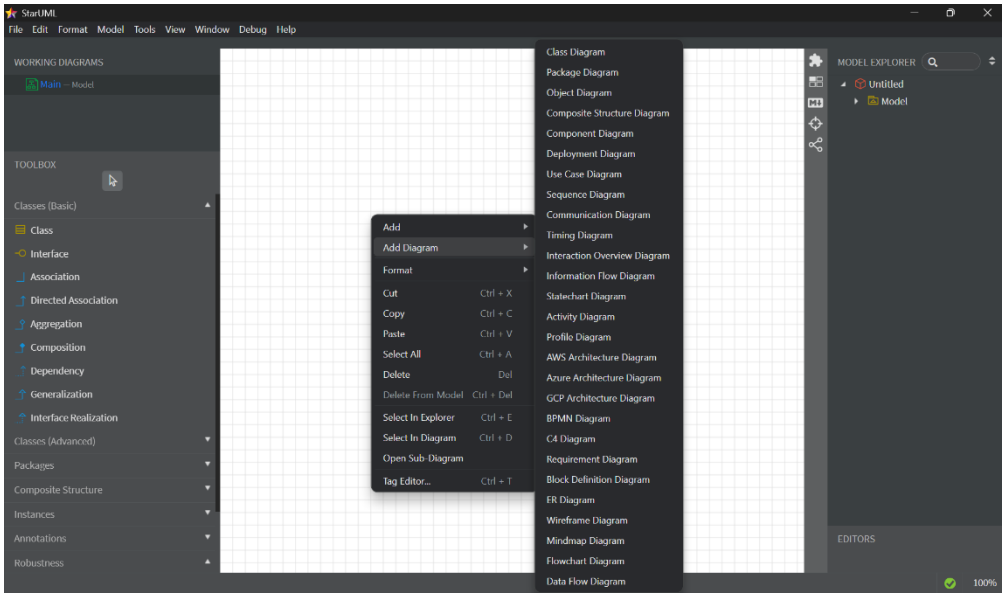

Рисунок 1.1 – Интерфейс главного окна *StarUml*

В *StarUML* можно создавать различные типы диаграмм, включая:

– диаграммы классов – используются для описания структуры классов и их отношений;

– диаграммы последовательностей – используются для описания взаимо действия между объектами и процессов, происходящих во времени;

– диаграммы состояний – используются для описания жизненного цикла объекта и его состояний;

– диаграммы компонентов – используются для описания структуры и отношений между компонентами системы;

– диаграммы развертывания – используются для описания физического размещения компонентов и системы в целом;

– диаграммы активностей – используются для описания последовательности действий и процессов в системе.

Данное приложение имеет множество функций, которые облегчают создание и редактирование диаграмм, такие как автоматическое выравнивание элементов, возможность группировки элементов, подсветка синтаксиса, подсказки и многое другое. Кроме того, *StarUML* позволяет пользователю создавать свои собственные элементы, шаблоны и плагины, чтобы улучшить функциональность и адаптировать инструмент под свои потребности.

1.5 Постановка задачи на дипломное проектирование

2 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Общая структура приложения

2.2 Функциональная модель программного комплекса

2.3 Информационная модель программного комплекса